7.4.5 Split transaction：高速 Hub 后面的低速设备

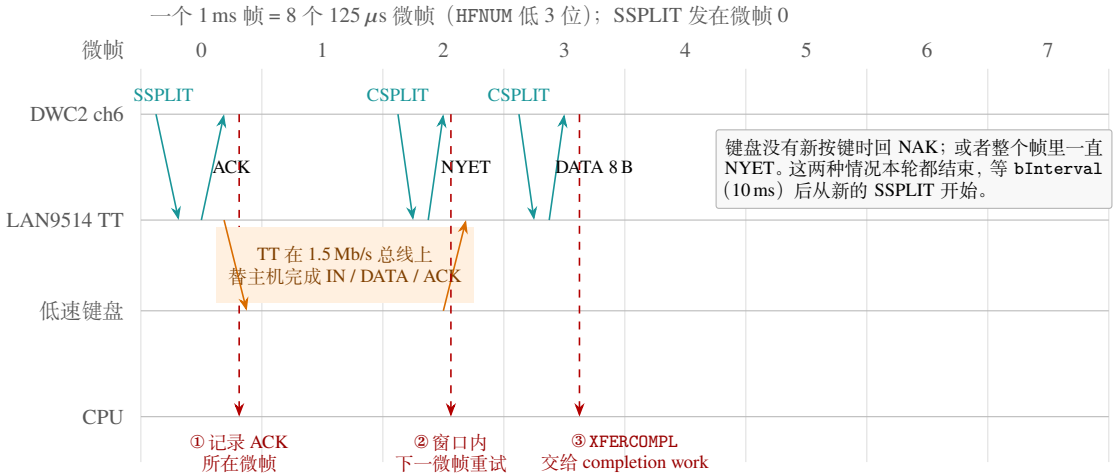


图 7.11: 低速键盘在高速 Hub 后面的一次 interrupt-IN 轮询。主机先向 TT 发 start-split，TT 在低速总线上替主机完成 IN 事务，主机再在随后的微帧里发 complete-split 取回结果。①、② 两种中间事件必须在中断里当场处理，晚一个帧就错过了窗口

DWC2 不能直接对低速设备说话，它要先把事务交给 Hub 里的 TT（图 7.11）：*start-split*（SSPLIT）告诉 TT“请替我对 port 3 上地址 4 的端点 1 做一次 IN”，TT 回 ACK 表示接受；TT 随后在低速总线上完成真正的事务；主机在随后的微帧里发 *complete-split*（CSPLIT）取结果，TT 要么回 NYET（还没完成），要么回数据，要么转达设备的 NAK。channel 寄存器里，这些信息体现为：

表 7.4: 真机日志中的寄存器样本解码

样本	含义
`HCCHAR=0x010e8808`	设备地址 4、端点 1、IN、interrupt 类型、低速（LSPDDEV）、最大包长 8
`HCSPLT=0x8001c083`	SPLTENA、COMPSPLT（当前是 CSPLIT）、XACTPOS=ALL、Hub 地址 1、端口 3
`HCINT=0x22`	`CHHLTD | ACK`：SSPLIT 已被 TT 接受
`HCINT=0x42`	`CHHLTD | NYET`：CSPLIT 暂时没有结果
`HCINT=0x03`	`CHHLTD | XFERCOMPL`：DMA 缓冲里已经是完整的报告

这里有三条规则，每一条都是在真机上踩过才写进代码的：低速设备的每一个事务都要在 `HCCHAR` 里置 `LSPDDEV`；CSPLIT 收到 NAK 说明这次事务已经结束（设备没数据），必须从新的 SSPLIT 重来，而不是继续发 CSPLIT；CSPLIT 收到 NYET 只能在同一个帧的窗口里重试，超出窗口就放弃本轮、等 `bInterval` 后重新开始。

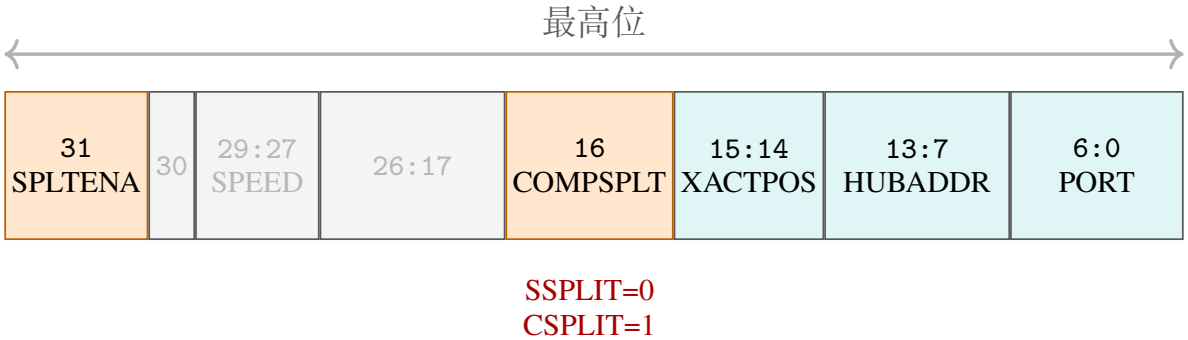


图 7.12: `HCSPLT` 的位域。只要 `SPLTENA=` 1，这次 channel 事务就不由 DWC2 自己驱动总线，而是交给 `HUBADDR+PORT` 指定的那颗 Hub 事务转换器（TT）代跑；两拍之间只有 `COMPSPLT` 一位不同（SSPLIT 为 0、CSPLIT 为 1），日志里 `0x8000c083` 与 `0x8001c083` 正是这一位之差。`XACTPOS` 本项目恒取 ALL（一次搬整包，不做首/尾微帧的分段）